



Ebike-Display auf ESP32-S3 mit reflektivem LCD

Projektbericht — Stand August 2026

RadDisplay

Entstehung dieses Projekts

Konzeption und Umsetzung entstanden in gemeinsamer Arbeit mit der KI **Claude** von Anthropic. Die Zusammenarbeit verlief dabei als fortlaufender Dialog und nicht als reine Codegenerierung: Hardwareauswahl, Architekturentscheidungen und Layoutfragen wurden diskutiert, Zwischenstände als Screenshots zurückgespielt und daraufhin gemeinsam nachgebessert.

Auch das Übersichtsschema in Abschnitt 3 entstand auf diesem Weg, erstellt mit Claude Design.

Gerade dieser Rückkopplungskreis war für das Ergebnis wesentlich. Mehrere Detailprobleme — die Kollision des Kilometerstands mit der Skala, die zu nah an ihren Strichen stehenden Skalenzahlen, die ausgefranstes Kanten schräger Linien — wurden erst anhand der Bildschirmfotos aus dem laufenden Simulator erkannt und anschließend behoben. Auch Fehler auf Seiten der KI, etwa ein versehentlich doppelt eingefügter Codeblock, traten dabei zutage und wurden korrigiert.

1. Ausgangslage und Zielsetzung

Ziel des Projekts ist ein Fahrradcomputer im Stil klassischer Motorrad- und Fahrzeuginstrumente: ein analoger Rundtacho mit Zeiger und Skalenkranz, ergänzt um Uhrzeit, Kilometerstand, digitale Geschwindigkeitsanzeige, Blinkerpfeile und eine Spalte mit Warnsymbolen. Vorbild war ein Retro-Layout im Pixellook, wie es sich mit rein schwarz-weißen Displays besonders überzeugend umsetzen lässt.

Der bewusste Verzicht auf Graustufen und Farbe ist dabei kein Kompromiss, sondern Teil des gestalterischen Konzepts. Harte Pixelkanten wirken auf einem monochromen Reflexivdisplay eigenständig und gut lesbar, während weichgezeichnete Grafiken dort unsauber erscheinen.

2. Hardware

Als Plattform dient das Waveshare ESP32-S3-RLCD-4.2. Das Board vereint einen ESP32-S3 mit 8 MB Octal-PSRAM und ein 4,2 Zoll großes reflektives LCD mit 400 × 300 Pixeln, angesteuert über einen ST7305-Controller per SPI. Die Anzeige arbeitet rein schwarz-weiß und ohne Hintergrundbeleuchtung — sie nutzt das Umgebungslicht, ähnlich einem E-Paper, erreicht aber deutlich höhere Bildwiederholraten.

Für den Einsatz am Fahrrad bringt das Board erhebliche Vorteile mit: Der Kontrast steigt bei direkter Sonneneinstrahlung, statt wie bei herkömmlichen LCDs einzubrechen. Der Stromverbrauch liegt spürbar unter dem eines hintergrundbeleuchteten Displays.

Eine Einschränkung muss man kennen: Ohne Beleuchtung ist bei Dunkelheit nichts ablesbar. Für Nachtfahrten wäre eine externe Anleuchtung nötig — konstruktiv lösbar, aber ein Punkt, der in die Gehäuseplanung gehört.

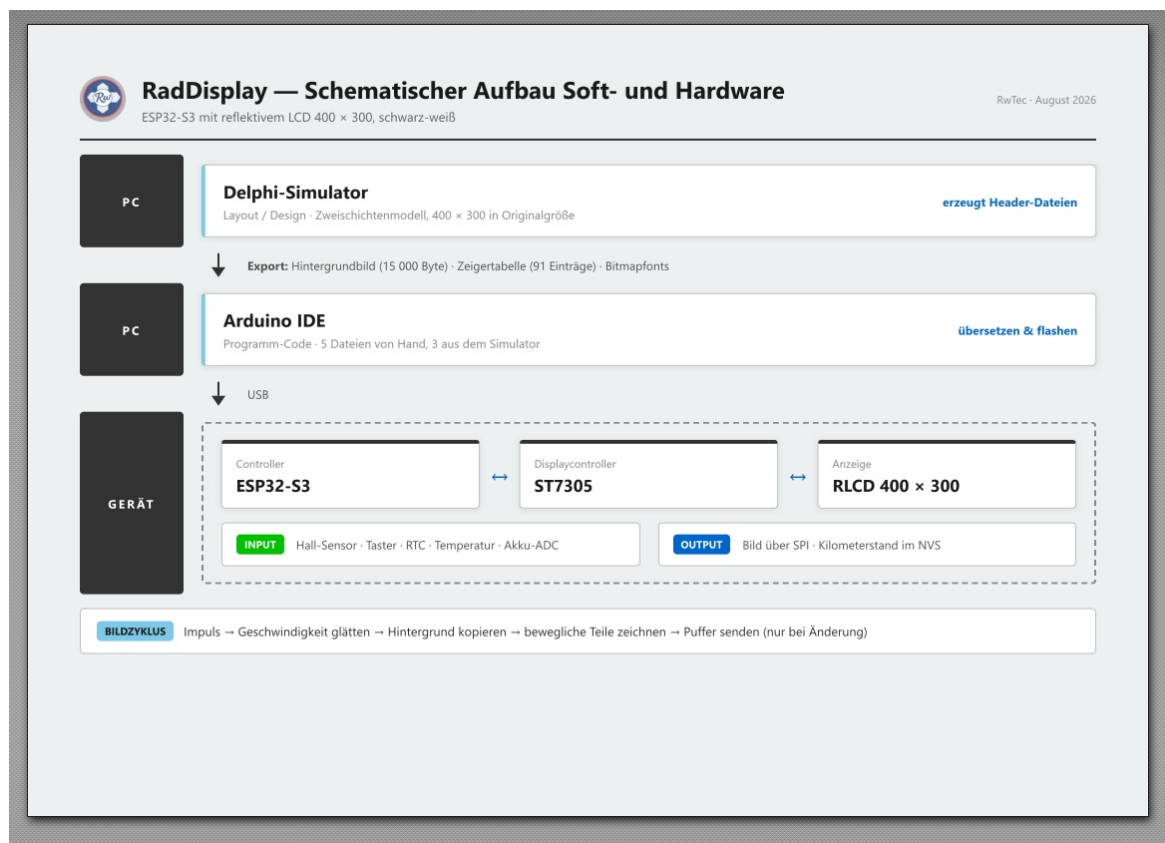
Weitere bereits vorhandene Komponenten des Boards werden im Projekt genutzt:

Baustein	Funktion im Projekt
PCF85063 RTC	Uhrzeitanzeige
SHTC3	Temperaturüberwachung, Glättewarnung
ADC an GPIO4	Akkuüberwachung über 3-fach-Spannungsteiler
Taster an GPIO18	Umschaltung der Lichtanzeige
Stiftleiste	Anschluss des Hall-/Reedsensors an GPIO17

Die Geschwindigkeitserfassung erfolgt klassisch über einen einzelnen Magneten an der Speiche und einen Sensor am Rahmen. Der gemessene Radumfang beträgt 2160 mm.

Der 18650-Halter des Boards ist für den Fahrradbetrieb nicht die naheliegende Lösung — sinnvoller wäre eine Versorgung über einen DC/DC-Wandler aus dem Hauptakku des Rades.

3. Grundgedanke der Softwarearchitektur



Der gesamte Weg vom Entwurf bis zur Anzeige: Auf dem PC entsteht im Delphi-Simulator das Layout, das als Header-Dateien in die Arduino-Entwicklungsumgebung übergeht. Von dort gelangt das übersetzte Programm über USB auf den ESP32-S3, der die Sensorik ausliest und das fertige Bild über SPI an den ST7305 sendet.

Der zentrale Entwurfsgedanke ist die Trennung des Bildinhalts in zwei Ebenen.

Die **statische Ebene** enthält alles, was sich während des Betriebs nie ändert: Doppelrahmen, Skalenstriche, Skalenzahlen, die Einheit, den Schriftzug im Zifferblatt, den Text in der oberen rechten Ecke sowie die vier Symbolkästen in ihrem Ruhezustand. Diese Ebene wird einmal erzeugt und liegt anschließend als 15000 Byte großes Array im Flash des ESP32.

Die **dynamische Ebene** umfasst die beweglichen Elemente: Zeiger, Uhrzeit, Kilometerstand, digitale Geschwindigkeit sowie die Zustandswechsel von Blinkerpfeilen und Warnsymbolen.

Der Ablauf pro Bild ist damit denkbar einfach: Hintergrund in den Arbeitspuffer kopieren, bewegliche Teile darüberzeichnen, Puffer ans Display senden.

Der eigentliche Kunstgriff betrifft die Symbole. Sie liegen im Hintergrundbild in ihrer inaktiven Darstellung — weißer Kasten mit schwarzem Symbol. Der aktive Zustand aus dem Vorbild, also ein schwarzer Kasten mit weißem Symbol, entsteht durch schlichtes Umkehren aller Pixel im betreffenden Rechteck. Der Mikrocontroller braucht dadurch keine einzige Zeichenroutine für Batterie, Scheinwerfer, Warndreieck oder Thermometer. Aus vier verhältnismäßig aufwendigen Grafikfunktionen wird eine einzige, wenige Zeilen lange Invertierung.

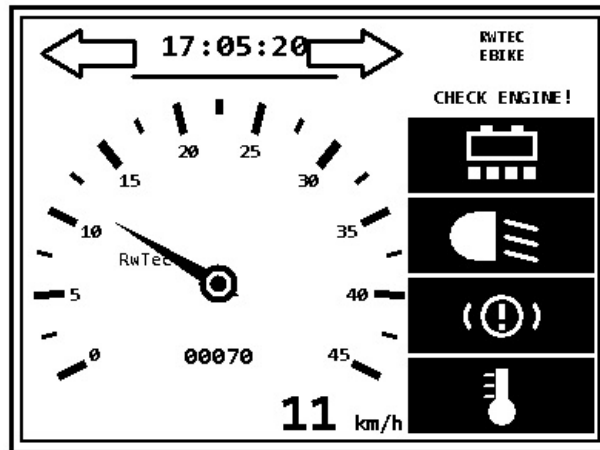
Analog dazu steht der Warntext dauerhaft im Hintergrund und wird bei Bedarf weiß übermalt. Die Blinkerpfeile liegen als Umriss im Hintergrund; im aktiven Zustand füllt der Controller das zugehörige Polygon aus, dessen sieben Eckpunkte ebenfalls aus dem Entwurf stammen.

4. Der Delphi-Layoutsimulator

4.1 Aufbau

Für den Layoutentwurf entstand eine eigenständige VCL-Anwendung unter Delphi 12.1. Sie besteht aus drei Units: einer reinen Renderer-Unit ohne Oberflächenbezug, einer Export-Unit und dem Hauptformular, das vollständig im Quelltext aufgebaut wird und bewusst ohne Formulardatei auskommt.

Die Anwendung zeichnet das Display in Originalgröße von 400 × 300 Pixeln, ausschließlich in Schwarz und Weiß und mit abgeschaltetem Antialiasing, und zeigt es zweifach vergrößert an. Ein Schieberegler steuert die Geschwindigkeit, ein Demomodus lässt den Zeiger selbstständig pendeln, Kontrollkästchen schalten Blinker und Warnsymbole.



Das fertige Layout in der Vorschau des Delphi-Simulators — 400 × 300 Pixel, rein schwarz-weiß, wie es später auf dem Reflexivdisplay erscheint.

Entscheidend ist, dass die Anwendung intern exakt dasselbe Zweischichtenmodell verwendet wie später der Mikrocontroller: Sie hält den statischen Hintergrund als eigenes Bitmap, kopiert ihn vor jedem Bildaufbau und zeichnet die beweglichen Teile darüber. Was in der Vorschau zu sehen ist, entspricht damit nicht nur optisch, sondern auch verfahrenstechnisch dem Zielgerät.

4.2 Vorteile dieses Vorgehens

Entwurfsgeschwindigkeit. Der Unterschied ist erheblich. Eine Layoutänderung am Mikrocontroller bedeutet Kompilieren, Flashen, Gerät beobachten — je nach Projektgröße eine halbe bis mehrere Minuten pro Iteration. In Delphi sind es wenige Sekunden. Bei der Feinabstimmung eines Zifferblatts, wo es um einzelne Pixel geht, summiert sich das schnell auf Stunden.

Sichtbarkeit von Kollisionen. Mehrere Probleme wurden erst in der Vergrößerung sichtbar und ließen sich sofort beheben: Der Kilometerstand kollidierte mit einer Skalenzahl und wanderte daraufhin unter die Zeignabe — den einzigen Bereich innerhalb des Zifferblatts, den der Zeiger konstruktionsbedingt nie überstreicht, da er senkrecht nach unten nie zeigt. Die Skalenzahlen berührten ihre eigenen Striche, weil breite Zahlen am unteren Bogenende seitlich in den Strich hineinragten. Der Zeiger war zu lang und schnitt bei mittleren Geschwindigkeiten in die Beschriftung. Die Symbolspalte stieß an den Rahmen.

Qualität der Grafikprimitive. Ein Detail, das ohne visuelle Kontrolle vermutlich unbemerkt geblieben wäre: Windows setzt bei schrägen Linien mit einer Strichstärke über einem Pixel die Endkappen unsauber. Auf einem monochromen Display ohne Kantenglättung ist jede ausgefrante Kante deutlich sichtbar. Die Skalenstriche werden deshalb nicht als Linien, sondern als gefüllte Vierecke gezeichnet, deren Eckpunkte selbst berechnet werden.

Verlagerung der Rechenarbeit. Alles, was sich vorab bestimmen lässt, wird auf dem PC bestimmt. Der Mikrocontroller führt zur Laufzeit keine einzige trigonometrische Berechnung aus.

Eine einzige Quelle für alle Koordinaten. Sämtliche Geometrie — Symbolrechtecke, Warntextrechteck, Pfeilpolygone, Ankerpunkte für Uhr, Kilometerstand und Digitalanzeige, Mittelpunkt und Radien des Zifferblatts — wird mitexportiert. Im Sketch steht keine einzige Koordinate doppelt. Eine Layoutänderung in Delphi wirkt sich nach einem Neuexport automatisch auf beiden Seiten aus, ein Auseinanderdriften von Vorschau und Gerät ist konstruktiv ausgeschlossen.

Einheitliche Typografie. Auch die beweglichen Ziffern werden aus Delphi exportiert. Andernfalls würden feste Skalenzahlen und bewegte Anzeigen unterschiedliche Schriftarten verwenden — ein Bruch, der sofort auffällt.

4.3 Die Exportkette

Drei Schaltflächen erzeugen die Übergabedateien:

Der **Hintergrundexport** liefert die statische Ebene als 15000-Byte-Array im Format 50 Byte je Zeile, MSB links, gesetztes Bit gleich schwarz. Beigefügt sind sämtliche Layoutkonstanten sowie das vorausberechnete Rechteck, das der Zeiger maximal überstreicht.

Die **Zeigertabelle** enthält für jede halbe km/h zwischen 0 und 45 die vier Eckpunkte des Zeigerpolygons — insgesamt 91 Einträge.

Der **Ziffernexport** rendert die Ziffern 0 bis 9 in drei Größen als Bitmapfont: 19 × 41 Pixel für die große Digitalanzeige, 12 × 24 Pixel einschließlich Doppelpunkt für die Uhr, 9 × 19 Pixel für den Kilometerstand.

Eine vierte Schaltfläche speichert die aktuelle Vorschau als Bilddatei.

4.4 Aufbau des Delphi-Projekts

Der Simulator ist eine gewöhnliche VCL-Anwendung unter Delphi 12.1 und besteht aus vier Dateien. Zusätzliche Komponenten oder Bibliotheken werden nicht benötigt; verwendet wird ausschließlich die Standard-VCL.

Datei	Zweck
EbikeSim.dpr	Projektdatei. Bindet die drei Units ein und erzeugt das Hauptformular.
uEbikeRender.pas	Der eigentliche Renderer und das Herzstück des Projekts. Enthält die vollständige Layoutbeschreibung — Drehpunkt, Radien, Skalenendwert, Winkelbereich, Ankerpunkte —, die Zeichenroutinen für Skala, Zeiger, Symbole und Blinkerpeile sowie die Trennung in statische und dynamische Ebene. Kennt keine Oberfläche und keine Steuerelemente, sondern zeichnet ausschließlich auf ein übergebenes Bitmap.
uEbikeExport.pas	Erzeugt aus dem gezeichneten Bild und der Layoutbeschreibung den C-Quelltext für den Mikrocontroller: Packen des Bildes nach einem Bit je Pixel, Ausgabe als Array, Berechnung der Zeigertabelle, Ableitung sämtlicher Geometriekonstanten und Rendern der Ziffern als Bitmapfont.
uEbikeMain.pas	Hauptformular mit Vorschau, Schieberegler, Kontrollkästchen und den Export-Schaltflächen. Das Formular wird vollständig im Quelltext aufgebaut; eine Formulardatei existiert bewusst nicht.

Der Verzicht auf eine `.dfm`-Datei ist eine bewusste Entscheidung. Die Oberfläche besteht aus rund einem Dutzend Steuerelementen ohne gestalterischen Anspruch — sie im Quelltext anzulegen kostet kaum mehr Aufwand, macht das Projekt aber unabhängig vom Formulardesigner und damit leichter zwischen Delphi-Versionen übertragbar. Beim Anlegen des Projekts ist deshalb das automatisch erzeugte Formular samt seiner Unit zu entfernen.

Auch hier gilt das Schichtenprinzip: `uEbikeRender` weiß nichts vom Export, `uEbikeExport` nichts von der Oberfläche. Eine Layoutänderung betrifft daher immer nur die Renderer-Unit, und zwar meist nur die Funktion mit den Vorgabewerten des Zifferblatts.

Zwei Stellschrauben sind für spätere Anpassungen besonders relevant. Die Vorgabewerte der Layoutbeschreibung fassen die gesamte Geometrie des Zifferblatts an einer Stelle zusammen — Skalenendwert, Radien und Zeigerlänge lassen sich dort in Sekunden ändern. Und eine Konstante für den Schriftnamen bestimmt die Typografie des gesamten Displays; für den Retro-Look genügt es, dort einen installierten Pixelfont einzutragen.

5. Implementierung auf dem ESP32-S3

5.1 Grafikschicht

Der Arbeitspuffer umfasst 15000 Byte und liegt im internen RAM. Implementiert sind Pixelzugriff, Rechteckfüllung, Rechteckinvertierung, Kreisfüllung sowie eine Polygonfüllung nach dem Scanline-Verfahren. Letztere wird sowohl für den Zeiger als auch für die Blinkerpeile verwendet. Die Textausgabe arbeitet ausschließlich mit den exportierten Bitmapfonts.

5.2 Geschwindigkeitsmessung

Dieser Teil verdient besondere Aufmerksamkeit, weil ein einzelner Magnet eine grundsätzliche Schwierigkeit mit sich bringt: Die Messwerte kommen selten. Bei 45 km/h vergehen zwischen zwei Impulsen 173 Millisekunden, bei 10 km/h bereits 778 Millisekunden, bei Schrittgeschwindigkeit über anderthalb Sekunden.

Zwei Konsequenzen ergeben sich daraus. Erstens würde eine Anzeige, die nur bei Impulsen aktualisiert, ruckartig springen. Dem begegnet eine exponentielle Glättung, die den Zeiger zwischen den Messungen weiterwandern lässt.

Zweitens — und gravierender — würde die Anzeige beim Anhalten auf dem letzten gemessenen Wert einfrieren, da schlicht kein weiterer Impuls eintrifft. Die Lösung besteht darin, nicht nur die letzte vollständige Umdrehung auszuwerten, sondern auch die seit dem letzten Impuls verstrichene Zeit. Ist diese bereits länger als die vorherige Umdrehung, kann das Rad nicht schneller geworden sein. Die Anzeige wird dann anhand der verstrichenen Zeit nach unten gerechnet und fällt von selbst gegen null. Nach vier Sekunden ohne Impuls gilt das Rad als stehend.

Im Interrupt sorgt eine Sperrzeit von 50 Millisekunden für die Unterdrückung des Kontaktprellens. Dieser Wert entspräche rechnerisch 155 km/h — was schneller eintrifft, kann nur Prellen sein.

5.3 Kilometerzähler

Der Zähler summiert Impulse und rechnet sie über den Radumfang in Meter um. Alle 100 gefahrene Meter wird der Stand im nichtflüchtigen Speicher abgelegt. Dieses Intervall ist ein bewusster Kompromiss zwischen Genauigkeit nach einem Stromausfall und Schonung des Flash-Speichers.

5.4 Peripherie und Anzeigelogik

Die Uhrzeit wird alle zwei Sekunden aus dem RTC gelesen; ein gemeldeter Oszillatorausfall führt zur Ersatzanzeige und aktiviert gleichzeitig das Störungssymbol. Der Akkustand ergibt sich aus der ADC-Messung, umgerechnet auf den Bereich zwischen 2,5 und 4,2 Volt. Die Temperatur dient der Glättewarnung unterhalb von 3 °C und einer Hitzewarnung oberhalb von 45 °C.

Das Bild wird alle 200 Millisekunden neu aufgebaut, allerdings nur dann tatsächlich übertragen, wenn sich am Inhalt etwas geändert hat. Die Übertragung ist der mit Abstand aufwendigste Teil eines Bildzyklus, und unnötige Übertragungen kosten Strom — genau die Ressource, deretwegen dieser Displaytyp gewählt wurde.

5.5 Ansteuerung des ST7305

Die Anbindung des Displaycontrollers ist abgeschlossen. Grundlage war das Beispielprojekt von Waveshare, genauer dessen Displaymodul aus der LVGL-Fassung. Es lieferte zwei Dinge, die sich nicht sinnvoll erraten lassen: die Initialisierungssequenz des Controllers und die Anordnung der Pixel in dessen Speicher.

Die Initialisierung wurde unverändert übernommen und lediglich von ESP-IDF auf die Arduino-SPI-Schnittstelle umgeschrieben. Sie umfasst rund zwanzig Registerzugriffe, darunter Spannungseinstellungen für den Booster, Gate-Spannungen und Zeitsteuerungstabellen. An der Reihenfolge solcher Sequenzen sollte man nicht rütteln.

Deutlich interessanter ist die Speicheranordnung, denn sie ist alles andere als naheliegend. Ein Byte fasst nicht acht nebeneinanderliegende Pixel zusammen, sondern einen Block aus zwei Spalten und vier Zeilen. Zusätzlich läuft die vertikale Achse rückwärts. Die Zuordnung lautet:

```
inv_y = 299 - y
index = (x ÷ 2) · 75 + (inv_y ÷ 4)
bit    = 7 - ( (inv_y mod 4) · 2 + (x mod 2) )
```

Diese Abbildung wurde vor der Umsetzung nachgerechnet und geprüft: Sie trifft alle 120000 Bildpunkte genau einmal und endet exakt bei Index 14999, füllt die 15000 Byte also lückenlos und überschneidungsfrei.

Die Umwandlung findet ausschließlich beim Senden statt. Der Arbeitspuffer behält sein einfaches zeilenweises Format, das mit dem Delphi-Export übereinstimmt — die Eigenheiten des Controllers bleiben auf eine einzige Funktion beschränkt. Die Schleife läuft dabei über die Zielbytes und holt sich die acht zugehörigen Quellpixel, wodurch die Divisionen entfallen, die eine pixelweise Umrechnung erfordern würde. Das Waveshare-Beispiel löst dieselbe Aufgabe über Nachschlagetabellen im PSRAM; bei 120000 Einträgen sind das mehrere hundert Kilobyte, was sich hier erübrigt, da ohnehin stets das Vollbild gewandelt wird.

Aus den Fensteradressen ließ sich außerdem eine für später nützliche Eigenschaft ablesen: Eine Speicherseite entspricht dem Wert $x \div 2$, und die 75 Byte einer Seite decken sämtliche 300 Bildzeilen ab. Ein partielles Update lässt sich in horizontaler Richtung daher beliebig fein eingrenzen, in vertikaler Richtung in Schritten von zwölf Zeilen.

Ein einziger Punkt bleibt offen und klärt sich erst am Gerät: welche Bitpolarität Schwarz bedeutet. Die zugehörige Farbkonstante steht in einer Header-Datei, die dem vorliegenden Beispielauszug nicht beilag. Für den Fall, dass das Bild als Negativ erscheint, ist ein Schalter vorgesehen, der die Umwandlung invertiert.

5.6 Aufbau des Sketch-Verzeichnisses

Das Projekt besteht in der Arduino-Entwicklungsumgebung aus acht Dateien. Sie müssen sich alle in einem gemeinsamen Ordner befinden, der genauso heißen muss wie die Hauptdatei — hier also `EbikeDisplay`. Zusatzdateien im Sketch-Ordner erscheinen nach dem Öffnen als eigene Reiter im Editor; fehlt einer davon, liegt die betreffende Datei am falschen Ort.

Drei der acht Dateien werden nicht von Hand geschrieben, sondern vom Delphi-Simulator erzeugt und nach jedem Layoutwechsel neu hineinkopiert. Sie sind in der folgenden Übersicht entsprechend gekennzeichnet.

Datei	Herkunft	Zweck
<code>EbikeDisplay.ino</code>	Hand	Hauptprogramm. Enthält die anpassbaren Einstellungen (Radumfang, Magnetzahl, Pinbelegung, Bildrate), die Interruptbehandlung des Hall-Sensors, die Geschwindigkeitsberechnung, den Kilometerzähler mit Ablage im nichtflüchtigen Speicher, das Auslesen von Uhr, Temperatursensor und Akkuspannung sowie den Bildaufbau samt Logik, welche Warnsymbole aktiv sind.
<code>gfx1bpp.h</code>	Hand	Schnittstelle der Grafikschrift: Framebuffer, Schriftbeschreibung und die Deklarationen der Zeichenfunktionen.
<code>gfx1bpp.cpp</code>	Hand	Umsetzung der Grafikschrift. Pixelzugriff, Rechteckfüllung und -invertierung, Kreisfüllung, Polygonfüllung nach dem Scanline-Verfahren sowie die Textausgabe mit den exportierten Bitmapfonts. Arbeitet ausschließlich auf dem zeilenweisen Framebuffer und kennt den Displaycontroller nicht.
<code>rlcd_hal.h</code>	Hand	Schnittstelle zum Display: Pinbelegung, SPI-Takt, Fensteradressen, Puffergröße und der Schalter für die Bitpolarität.
<code>rlcd_hal.cpp</code>	Hand	Ansteuerung des ST7305. Initialisierungssequenz, SPI-Sendefunktionen und die Umwandlung vom zeilenweisen Framebuffer in die blockweise Speicheranordnung des Controllers. Die einzige Datei, die die Eigenheiten der Anzeigehardware kennt.
<code>gauge_background.h</code>	Delphi-Export	Statische Bildebene als 15000-Byte-Array — Rahmen, Skala, Beschriftung, Symbole im Ruhezustand. Zusätzlich sämtliche Geometrie: Symbolrechtecke, Warntextrechteck, beide Pfeilpolygone, Ankerpunkte der beweglichen Texte und der vom Zeiger überstrichene Bereich.
<code>gauge_needle.h</code>	Delphi-Export	Vorausberechnete Zeigergeometrie. 91 Einträge für den Bereich 0 bis 45 km/h in Schritten von einer halben Stundenkilometer, je vier Eckpunkte.
<code>gauge_fonts.h</code>	Delphi-Export	Bitmapfonts der Ziffern in drei Größen: 19 × 41 Pixel für die Digitalanzeige, 12 × 24 Pixel einschließlich Doppelpunkt für die Uhr, 9 × 19 Pixel für den Kilometerstand.

Zusätzliche Bibliotheken sind nicht zu installieren. Verwendet werden ausschließlich `SPI`, `Wire` und `Preferences`, die alle zum ESP32-Kern gehören.

Die Aufteilung folgt einem einfachen Gedanken: Jede Schicht kennt nur die darunterliegende. Das Hauptprogramm weiß nichts von SPI, die Grafikschrift nichts vom Displaycontroller, und die Controlleranbindung nichts vom Tacho. Ein Wechsel auf ein anderes Display beträfe deshalb allein `rlcd_hal.cpp`.

Praktischer Hinweis für den laufenden Betrieb: Die Export-Schaltflächen des Simulators legen ihre Dateien neben der ausführbaren Datei ab, also im Unterverzeichnis der Übersetzungsausgabe. Von dort müssen sie in den Sketch-Ordner kopiert werden. Wer das häufiger tut, kann im Simulator den Zielpfad fest auf den Sketch-Ordner setzen und sich den Zwischenschritt sparen.

6. Offene Punkte

Der gesamte Softwarestand ist fertiggestellt und übersetzt fehlerfrei. Die Anzeigehardware ist zum Berichtszeitpunkt noch nicht beschafft, weshalb sämtliche Erprobung aussteht.

Inbetriebnahme des Displays. Der erste Prüfschritt ist die Darstellung des reinen Hintergrundbildes, das bereits während der Initialisierung übertragen wird. Das Fehlerbild ist dabei aussagekräftig: Erscheint die Darstellung als Negativ, ist die Bitpolarität umzuschalten. Erscheint sie um 90 Grad gedreht, liegt es am Register für die Speicherzugriffsrichtung. Erscheint sie gestreift oder zerhackt, stimmt die Formatumwandlung nicht.

Übersetzungsstand. Der Sketch kompiliert unter der Arduino IDE mit dem generischen Board-Profil ESP32S3 Dev Module. Eine Einstellung verdient Beachtung: Der PSRAM-Modus muss auf Octal-PSRAM stehen. In der Voreinstellung ist er abgeschaltet, was folgenlos durchkompiliert, das Board später aber nicht starten lässt. Für dieses Modul existiert kein eigener Board-Eintrag; die vorhandenen Waveshare-Profile gehören zu Touch-LCD- und AMOLED-Varianten mit abweichender Speicherbestückung und sind nicht verwendbar.

Prüfung der Sensorik. Der Sensor sollte zunächst am Schreibtisch mit einem von Hand geführten Magneten getestet

werden, bevor er ans Rad kommt. Zu beobachten sind Zeigerlauf, Abfallverhalten beim Stillstand und die Zuverlässigkeit der Prellunterdrückung.

Kalibrierung des Radumfangs. Der angesetzte Wert von 2160 mm ist ein guter Ausgangspunkt, sollte aber durch Abrollen überprüft werden. Der Wert geht unmittelbar in Tachoanzeige und Kilometerzähler ein.

Fahrpraktische Erprobung. Ablesbarkeit bei unterschiedlichen Lichtverhältnissen, Verhalten bei Erschütterung, Stromaufnahme im Dauerbetrieb.

Stromversorgung und Gehäuse. Der Wandler vom Hauptakku des Rades sowie ein wettergeschütztes Gehäuse mit Lenkerhalterung sind noch zu entwerfen.

Blinkereingänge. Die Auswertung ist im Quelltext vorbereitet, aber deaktiviert. Sobald feststeht, wie die Blinker angesteuert werden, sind lediglich zwei Pinnummern einzutragen.

7. Ausblick

Als nächster sinnvoller Ausbauschritt bietet sich das partielle Bildaktualisieren an. Statt bei jeder Änderung das vollständige Bild zu übertragen, würde nur der Bereich gesendet, in dem sich der Zeiger bewegt. Das zugehörige Rechteck wird bereits mitexportiert, und die Analyse der Fensteradressierung hat gezeigt, dass die Eingrenzung technisch möglich ist. Der Gewinn läge in geringerer Stromaufnahme und höherer möglicher Bildrate.

Darüber hinaus ist die Architektur offen für weitere Anzeigen — Trip-Kilometer, Durchschnitts- und Höchstgeschwindigkeit oder die Akkureichweite. Jede davon ist ein Eintrag im Delphi-Simulator, ein Neuexport und eine Textausgabe im Sketch.

Anhang: Vorbild der Gestaltung



Dieses Instrument gab die gestalterische Richtung vor: Rundtacho mit Zeiger und Skalenkranz, Uhrzeit mit Unterstrich, Blinkerpeile, eine Spalte invertierter Warnsymbole und der durchgehende Pixellook. Übernommen wurden Aufbau und Bildsprache, nicht die Werte — die Skala endet hier bei 45 statt 55 km/h, passend zum Einsatz am Fahrrad.

Hinweis: Die Aufnahme stammt aus einer fremden Veröffentlichung und dient hier ausschließlich der Veranschaulichung. Für eine Weitergabe des Berichts sollte sie durch ein eigenes Foto des fertigen Geräts ersetzt werden.



Erstellt von RwTec in Zusammenarbeit mit Claude (Anthropic), August 2026.